</>

SQL PORTFOLIO

RETAIL SALES PERFORMANCE ANALYSIS

MYSQL	30 QUERIES	WINDOW FUNCTIONS · CTEs · SUBQUERIES

Prepared by Fathima Suzain
Data Analytics Portfolio · 2026

00 PROJECT OVERVIEW

Objective

This project uses SQL to explore and analyze retail sales performance for a 3-month window (April–June 2026). The goal is to answer common business questions around revenue, customer behavior, delivery performance, and payment trends — progressing from basic filtering and aggregation to more advanced techniques such as window functions, subqueries, and CTEs.

Dataset

The dataset (**retail_sales** table) contains 140 retail orders from a retail apparel business, covering order details, customer demographics, product category, city, delivery status, payment method, price, quantity, revenue, and customer rating. This is the same underlying dataset used in the companion Excel dashboard project.

At a Glance

30	4	3	MySQL
Queries	Sections	Advanced techniques	Database

How This Document Is Organized

Queries are grouped into four sections that mirror a typical analysis workflow: **Data Exploration & Filtering**, **Aggregations & Grouping**, **Business Insight Queries**, and **Advanced SQL**. Each query includes a short explanation of what it does and why it's useful, followed by the runnable code.

01 DATA EXPLORATION & FILTERING

1 Total number of orders

A quick sanity check on dataset size before starting deeper analysis.

```mysql
SELECT COUNT(*) FROM retail_sales;
```

2 All orders currently in Processing

Surfaces orders that haven't shipped yet, useful for operational follow-up.

```mysql
SELECT *
FROM retail_sales
WHERE DeliveryStatus = 'Processing';
```

3 Count of Processing orders

Turns the previous result into a single summary number.

```mysql
SELECT DeliveryStatus, COUNT(*) AS Total_Processing
FROM retail_sales
WHERE DeliveryStatus = 'Processing'
GROUP BY DeliveryStatus;
```

4 Check for missing values in key columns

A basic data-quality check, confirms whether critical fields (customer name, revenue, payment method) have any nulls before relying on them for analysis.

```mysql
SELECT
    SUM(CASE WHEN CustomerName IS NULL THEN 1 ELSE 0 END) AS Missing_Names,
    SUM(CASE WHEN Revenue IS NULL THEN 1 ELSE 0 END) AS Missing_Revenue,
    SUM(CASE WHEN PaymentMethod IS NULL THEN 1 ELSE 0 END) AS Missing_Payment
FROM retail_sales;
```

5 Select specific columns only

Returns just Order ID, Customer Name, and Revenue, useful when a full row isn't needed.

```mysql
SELECT OrderID, CustomerName, Revenue
FROM retail_sales;
```

6 Orders placed in May

Filters the dataset down to a single month for month-over-month comparison.

```mysql
SELECT *
FROM retail_sales
WHERE Month = 'May';
```

7 Orders with rating above 4.5

Identifies the highest-satisfaction orders.

```mysql
SELECT *
FROM retail_sales
WHERE CustomerRating > 4.5;
```

8 Orders paid using UPI

Filters to a single payment method for closer inspection.

```mysql
SELECT *
FROM retail_sales
WHERE PaymentMethod = 'UPI';
```

9 Count of orders paid via UPI

Summarizes the previous filter into a single count.

```mysql
SELECT PaymentMethod, COUNT(*) AS Total_Payments
FROM retail_sales
WHERE PaymentMethod = 'UPI'
GROUP BY PaymentMethod;
```

10 Orders from Bangalore

Filters orders down to a single city.

```mysql
SELECT *
FROM retail_sales
WHERE City = 'Bangalore';
```

11 Products priced above Rs. 1000

Identifies higher-priced items in the catalog.

```mysql
SELECT *
FROM retail_sales
WHERE Price > 1000;
```

02 AGGREGATIONS & GROUPING

12 Total revenue generated

The headline number for the whole 3-month period.

```mysql
SELECT SUM(Revenue) AS Total_Revenue
FROM retail_sales;
```

13 Average customer rating

Gives an overall sense of customer satisfaction.

```mysql
SELECT AVG(CustomerRating) AS Average_Rating
FROM retail_sales;
```

14 Average customer rating, rounded

Same as above, rounded to 2 decimal places for cleaner reporting.

```mysql
SELECT ROUND(AVG(CustomerRating), 2) AS Average_Rating
FROM retail_sales;
```

15 Highest and lowest product price

Establishes the price range across the catalog.

```mysql
SELECT
    MAX(Price) AS Highest_Price,
    MIN(Price) AS Lowest_Price
FROM retail_sales;
```

16 Orders by delivery status

Breaks down all orders by their current delivery stage.

```mysql
SELECT DeliveryStatus, COUNT(*) AS Delivery_Status_Orders
FROM retail_sales
GROUP BY DeliveryStatus;
```

17 Orders by payment method

Breaks down all orders by how they were paid for.

```mysql
SELECT PaymentMethod, COUNT(*) AS Total_Orders
FROM retail_sales
GROUP BY PaymentMethod;
```

18 Revenue by city

Shows which cities generate the most revenue.

```mysql
SELECT City, SUM(Revenue) AS Revenue_CityWise
FROM retail_sales
GROUP BY City;
```

19 Revenue by category

Shows which product categories generate the most revenue.

```mysql
SELECT Category, SUM(Revenue) AS Revenue_CategoryWise
FROM retail_sales
GROUP BY Category;
```

20 Average rating by category

Checks whether certain product categories tend to satisfy customers more than others.

```mysql
SELECT Category, AVG(CustomerRating) AS Avg_Rating
FROM retail_sales
GROUP BY Category;
```

2 1 Total quantity ordered by category

Measures demand volume per category, independent of price.

```mysql
SELECT Category, SUM(Qty) AS Total_Quantity_Ordered
FROM retail_sales
GROUP BY Category;
```

03 BUSINESS INSIGHT QUERIES

2 2 Top 5 cities by revenue

Ranks cities to identify the strongest markets.

```mysql
SELECT City, SUM(Revenue) AS Top_Revenues
FROM retail_sales
GROUP BY City
ORDER BY Top_Revenues DESC
LIMIT 5;
```

2 3 Top 5 categories by revenue

Ranks product categories to identify top performers.

```mysql
SELECT Category, SUM(Revenue) AS Top_Revenues_Categories
FROM retail_sales
GROUP BY Category
ORDER BY Top_Revenues_Categories DESC
LIMIT 5;
```

2 4 Month with the highest revenue

Identifies the strongest month across the 3-month window.

```mysql
SELECT Month, SUM(Revenue) AS Highest_Revenue
FROM retail_sales
GROUP BY Month
ORDER BY Highest_Revenue DESC
LIMIT 1;
```

25 Most used payment method

Identifies the single most common way customers pay.

```mysql
SELECT PaymentMethod, COUNT(PaymentMethod) AS Most_Used_PayMethod
FROM retail_sales
GROUP BY PaymentMethod
ORDER BY Most_Used_PayMethod DESC
LIMIT 1;
```

26 Classify orders as High / Medium / Low value

Uses a CASE statement to bucket every order into a value tier, useful for segmenting customers or orders by size.

```mysql
SELECT OrderID, Revenue,
    CASE
        WHEN Revenue > 5000 THEN 'HIGH VALUE'
        WHEN Revenue BETWEEN 2000 AND 5000 THEN 'MEDIUM VALUE'
        ELSE 'LOW VALUE'
    END AS Order_Category
FROM retail_sales;
```

04 ADVANCED SQL

27 Rank cities by revenue (window function)

Uses RANK() to assign each city a rank based on total revenue, without collapsing the result into just the top N like LIMIT would.

```mysql
SELECT City, SUM(Revenue) AS Total_Revenue,
    RANK() OVER (ORDER BY SUM(Revenue) DESC) AS City_Rank
FROM retail_sales
GROUP BY City;
```

28 Running total of revenue by date

Uses a window function to build a cumulative revenue total as orders progress chronologically, useful for tracking growth over time.

```mysql
SELECT
    OrderID,
    Date,
    Revenue,
    SUM(Revenue) OVER (ORDER BY Date, OrderID) AS Running_Total
FROM retail_sales;
```

29 Customers spending above the overall average (subquery)

Uses a subquery to compute the average order value, then filters customers against it, a common pattern for identifying above-average spenders.

```mysql
SELECT CustomerID, CustomerName, Revenue
FROM retail_sales
WHERE Revenue >
    (SELECT AVG(Revenue) FROM retail_sales)
ORDER BY Revenue DESC;
```

30 Top 3 cities and their category-wise revenue (CTE)

Uses a Common Table Expression (CTE) to first identify the top 3 cities by revenue, then joins back to the main table to break down each of those cities' revenue by product category.

```mysql
WITH top_cities AS (
    SELECT City, SUM(Revenue) AS Total_Revenue
    FROM retail_sales
    GROUP BY City
    ORDER BY Total_Revenue DESC
    LIMIT 3
)
SELECT s.City, s.Category, SUM(s.Revenue) AS Category_Revenue
FROM retail_sales s
JOIN top_cities t ON s.City = t.City
GROUP BY s.City, s.Category
ORDER BY s.City, Category_Revenue DESC;
```

End of document. Full runnable script available in the accompanying .sql file.